

0.31. Deux lettres à la fois

Le 0.30, plus une deuxième lettre : deux `un_pas` dans la même boucle, et les deux bougent ensemble.

1. Ce qu'il faut comprendre

C'est le 0.30, plus une lettre, le B, qui descend pendant que le A avance.

Ce qui est nouveau ici : deux appels à `un_pas` dans la même boucle, l'un après l'autre. Comme `un_pas` n'attend pas, les deux se font **dans la même image** : les deux lettres bougent **ensemble**. Avec `deplace`, le B aurait attendu la fin du trajet du A.

Chaque lettre a ses deux variables : `xa`, `ya` pour le A ; `xb`, `yb` pour le B.

Le B s'arrête en ligne 16 : le `if (yb < 16)` ne le laisse plus avancer après, pour qu'il ne touche pas les nombres de la ligne 17.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <un_pas>` → [0.29.4. La fonction un_pas\(\) — un seul pas, sans attendre](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1175 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	72 o	← la console, à l'allumage — la mise en route de la console
<code>un_pas()</code>	117 o	← <code>main()</code> — une fonction fournie par le langage
<code>main()</code>	131 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
<code>AttendreImage</code>	28 o	← <code>image()</code> — attend l'image suivante
<code>AttendreVBlank</code>	16 o	← démarrage — attend que l'écran soit libre
<code>AttendreAcces</code>	12 o	← <code>EffacerCases</code> , <code>EcrireNombre</code> — attend de pouvoir écrire une case
<code>AdresseCase</code>	10 o	← <code>effacer()</code> , <code>poser()</code> — trouve l'adresse d'une case de l'écran
<code>EffacerCases</code>	12 o	← <code>effacer()</code> — efface des cases
<code>EcrireNombre</code>	33 o	← <code>nombre()</code> — écrit un nombre en chiffres
<code>CopierTuiles</code>	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
<code>EffacerCarte</code>	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← <code>CopierTuiles</code> — la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code>	× 3	39 o sur place, dans <code>un_pas()</code> , <code>main()</code> + <code>AdresseCase</code> (10 o)
<code>nombre()</code>	× 2	20 o sur place, dans <code>main()</code> + <code>EcrireNombre</code> (33 o)

```
effacer() × 1    15 o sur place, dans un_pas() + AdresseCase (10 o), EffacerCases (12 o)
image()  × 1     3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
un_pas() × 2    44 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <un_pas>      // fait faire un seul pas à une tuile
#include <nombre>      // écrit un nombre en chiffres
```

2. UNE VARIABLE — XA

```
uint8_t xa = 0;      // le A : sa colonne...
```

3. UNE VARIABLE — YA

```
uint8_t ya = 0;      // ...et sa ligne. Il file vers la DROITE.
```

4. UNE VARIABLE — XB

```
uint8_t xb = 0;      // le B : sa colonne...
```

5. UNE VARIABLE — YB

```
uint8_t yb = 2;      // ...et sa ligne. Il file vers le BAS.
```

6. UNE VARIABLE — IMAGES

```
uint8_t images = 0;  // le chronomètre, en images
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(xa, ya, ALPHABET[0]); // les deux lettres à leur place de départ
    poser(xb, yb, ALPHABET[1]); // ALPHABET[1] : la 2e lettre, B

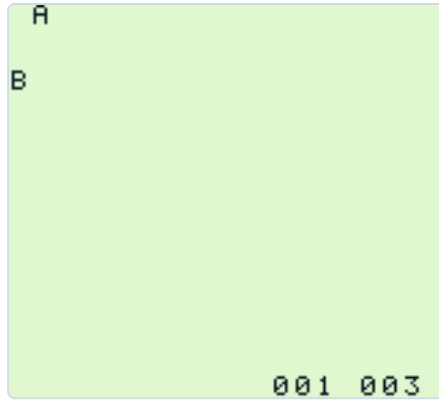
    while (true) {
        image();
        images++;
        if (images == 15) { // 4 fois par seconde :
            images = 0;
            un_pas(xa, ya, ALPHABET[0], 1, 0); // le A : un pas à droite
            // La ligne ajoutée : le B, un pas en bas – DANS LA MÊME IMAGE que le A,
            // puisque un_pas n'attend pas. Tant que yb < 16 seulement.
            if (yb < 16) un_pas(xb, yb, ALPHABET[1], 0, 1);
            nombre(12, 17, xa); // la colonne du A, pour la voir changer
            nombre(16, 17, yb); // la ligne du B
        }
    }
}
```

3. Ce qui se passe, étape par étape

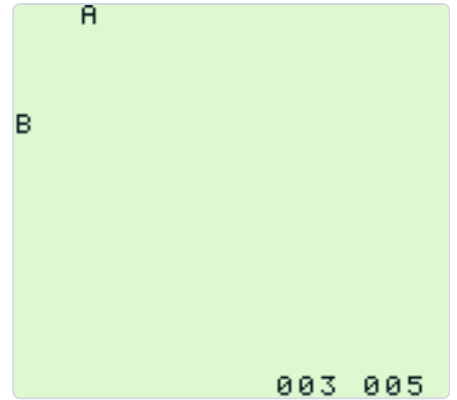
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



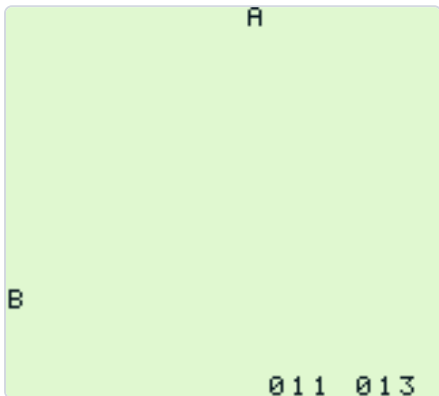
la toute première image — la console vient de démarrer — image 1



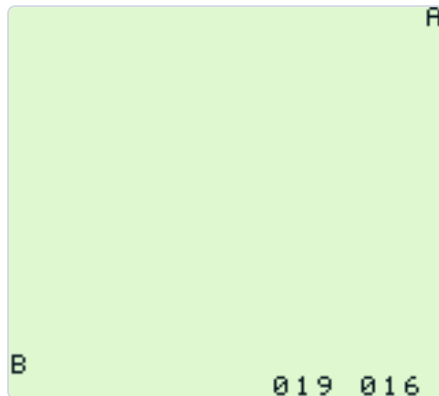
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui file vers la droite sur la ligne du haut, et un B qui descend en même temps sur la gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A et le B avancent (4 et 4)
- ✓ du même nombre de pas
- ✓ le A s'arrête au bord : colonne 19 ($x_a = 19$)
- ✓ le B s'arrête en ligne 16 ($y_b = 16$)

Ensuite : **0.31.1. Deux lettres à la fois — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++